

FinanceOS

AI 财务操作系统 · 产品设计文档

产品设计思路 · 用户需求分析 · 功能架构
AI 能力说明 · 落地可行性 · 商业化思考

项目代号：FinanceOS Desktop

文档版本：V1.0

日期：2026-08-06

文档性质：产品战略与方案说明

目录

1. 执行摘要
2. 产品设计思路
3. 用户需求分析
4. 功能架构
5. AI 能力说明
6. 落地可行性分析
7. 商业化思考
8. 路线图与风险
9. 结论与建议

1. 执行摘要

FinanceOS 将传统「财务审计 Web 系统」重构为「AI 财务操作系统」体验：登录后进入类 Windows 桌面，业务能力以独立应用窗口呈现，AI 作为系统级入口可调度应用、辅助分析与跨模块协作。

本项目不是自研操作系统内核，而是在 Web/桌面壳层构建「OS 隐喻」：快捷方式、开始菜单、任务栏、文件空间、终端、双内核浏览器，叠加财务核算、审计分析、协作与权限体系。该路径可显著降低用户对「又一个后台管理系统」的认知负担，同时保留 Flask 业务资产的可复用性。

当前已验证能力包括：Win11

风格桌面、核心快捷方式（仪表盘/表格编辑/AI/Files/Terminal/Browser）、角色权限过滤、AI 对话开应用、百度搜索代理、沙箱文件与终端、审计仪表盘与导入分析链路。

2. 产品设计思路

2.1 设计主张

「软件堆叠」→「系统工作台」。财务人员一天要在凭证、发票、对账、审计、沟通、搜资料之间切换；传统单页侧栏导航把所有功能压成菜单树，心智成本高。FinanceOS 采用操作系统隐喻：每个能力是一个 App，桌面是工作起点，AI 是调度中枢。

2.2 关键设计原则

品牌即系统：FinanceOS / AI 财务 OS，开始按钮使用自有 Logo，而非通用软件入口感

桌面优先：登录后是快捷方式桌面，不默认强弹 AI；AI 是可点开的系统应用

应用独立窗口：仪表盘、表格编辑等在 OS chrome 下隐藏侧栏，减少「网页套网页」噪音

AI 可调度：自然语言/语音打开应用；工作中可建议相关辅助应用（不强制打断）

权限仍在：岗位角色决定开始菜单可见业务，但核心桌面能力对登录用户可用，保证「像系统」的一致性

渐进真实感：先做高保真 Web 桌面，再可选 Electron/ISO；不幻想一期重写内核

2.3 体验层次

交互层（桌面壳）→ 应用层（财务/审计/协作 App）→ 智能层（Agent/模型/检索）→ 服务层（Flask API、权限、数据）→ 基础设施（Gunicorn、SQLite/可选 Meilisearch、用户沙箱文件）。

2.4 与「真 Linux 桌面 OS」的边界

战略上可对标「财务专用 Linux 桌面」愿景；工程上首期交付 Web OS +

可选客户端封装。明确不做：自研内核、驱动、完整发行版 ISO（可作为二期）。好处是业务价值可立刻验证，技术债可控，商业化可先卖「工作台订阅」而非「装机镜像」。

3. 用户需求分析

3.1 目标用户画像

角色	核心诉求	FinanceOS 对应
审计员 / 审计经理	快导入、看风险、出报告、留痕	仪表盘、Analysis、Report、History
会计 / 财务主管	凭证往来发票、校对数据	Vouchers、表格编辑、Import

出纳 / 税务助理	对账、发票、基础数据	Bank、Invoices、Files
顾问 / 学生	学习审计方法、只读分析	仪表盘、AI 讲解、知识检索
系统管理员	成员权限、模型与扩展	Company、Models、Agent Dev

3.2 痛点洞察

- 工具碎片化：Excel + 网盘 + IM + 多个财务网页，上下文丢失
- 审计门槛高：规则、Benford、函证等概念需要伴随式解释，而非冷冰冰菜单
- 协作成本高：改数、对口径、开会、传文件分散在不同软件
- 权限难讲清：同一公司多岗位，需要桌面一致 + 业务按权开放
- 搜索割裂：既要搜公网资料，也要搜项目内知识；浏览器与站内检索应并存

3.3 关键用户故事

- 作为审计员，我希望上传底稿后立刻在仪表盘看到风险概览，而不是找三个菜单
- 作为会计，我希望用表格编辑快速改错行，并让同事协同看同一张表
- 作为管理者，我希望对 AI 说「打开银行对账」，系统直接打开对应应用
- 作为新人，我希望在浏览器搜「函证」时，既能百度外网，也能进站内知识库

3.4 需求优先级（MoSCoW）

级别	需求项
Must	桌面壳、登录权限、Import、仪表盘、表格编辑、AI 开应用、基础审计规则
Should	Files/Terminal、Browser+搜索、Assist 建议、角色差异化应用、报告导出
Could	真双引擎浏览器、完整 Meilisearch、Electron 安装包、会议协作深化
Won't（首期）	自研 OS 内核、离线国产化整机镜像、替代用友/金蝶总账

4. 功能架构

4.1 总体架构

客户端为浏览器或可选 Electron。桌面壳负责窗口管理、快捷方式、开始菜单、任务栏 AI 命令。各 App 以 iframe/独立路由加载，业务 API 统一走 Flask。

4.2 功能域划分

功能域	模块示例	说明
系统壳	桌面、开始菜单、任务栏、Assist	OS 体验与应用生命周期
数据与编辑	Import、表格编辑、Preview	数据进入与校对
审计分析	仪表盘、Analysis、Report、History	风险发现与交付物
财务核算	Finance、Vouchers、应收应付、发票、对账	日常财务作业
智能与检索	AI、Models、Search、Browser	调度、问答、内外搜索
协作与治理	Chat、Company、Settings、RBAC	人与权限

本地空间	Files、Terminal、userspace	沙箱文件与命令
------	--------------------------	---------

4.3 桌面信息架构

默认桌面核心快捷方式置顶：仪表盘、表格编辑、AI、Files、Terminal、Browser、Import。开始菜单按 Category 分组，支持搜索与列表视图。

4.4 关键技术组件

- 后端：Flask + Gunicorn（线程模型），SQLite 业务与历史库
- 前端：原生 JS 桌面壳 + Jinja 模板业务页；?chrome=os 极简窗
- 检索：优先 Meilisearch，回退 SQLite FTS5 + BM25
- 浏览器：Blink iframe + Gecko 服务端抓取净化；关键词→百度（代理渲染）
- AI：/api/agent/chat + 客户端 actions（open_app / navigate）
- 权限：roles 功能开关 + 公司功能包交集

5. AI 能力说明

5.1 能力分层

层级	能力	产品价值
系统调度层	自然语言/语音打开应用、映射别名	降低导航成本，形成「OS 级助理」心智
工作辅助层	打开业务时建议相关 App（Assist）	减少漏看关联模块；避免强制打断
分析解释层	结合审计结果、知识条目问答	新人培训与报告解读
工具执行层	Agent tools / 工作流 / 扩展	可编排的自动化（持续建设）
模型管理层	本地/云端模型配置、策略	可控成本与数据边界

5.2 交互设计要点

- AI 是桌面应用而非强制首页，避免「一登录就被对话框占满」
- 任务栏常驻短命令框，适合「打开某某」高频意图
- Agent 在 OS iframe 内通过 postMessage 开窗，而不是整页跳转打断桌面
- Assist 默认仅建议、不自动连环开窗（防止弹窗风暴与误开 AI）

5.3 当前边界（诚实披露）

- 「双内核浏览器」为体验层双路径，并非嵌入完整 Chromium/Gecko 引擎二进制
- 百度结果依赖服务端抓取，受网络、反爬与页面结构影响
- Agent 能力取决于已配置模型与工具权限，不能承诺替代持证注册会计师判断
- AI 输出需人工复核，尤其涉及审计结论与对外报告

5.4 数据与安全原则

用户文件落在 data/userspace/{uid}/ 沙箱；终端命令白名单；外链浏览限制本机地址；业务权限与公司功能包约束。商业化场景应补充：租户隔离加固、审计日志、密钥托管、模型数据不出域选项。

6. 落地可行性分析

6.1 为何可行

- 存量资产复用：原财务审计 Flask 应用可直接挂到桌面 App，改造成本主要在壳与路由体验
- 交付形态灵活：同一后端可服务公网演示、内网部署、Electron 桌面包装
- 团队技能匹配：Web 全栈即可推进，无需内核/驱动团队
- 验证闭环短：桌面快捷方式与开应用链路已可演示，利于融资/客户试用

6.2 工程可行性

维度	评估	说明
技术成熟度	高	Flask/Pandas/前端桌面壳均为成熟栈
性能	中高	Gunicorn 多线程；重分析需控制并发与缓存
安全	中	已有沙箱与 RBAC；商业版需渗透测试与等保对齐
运维	高	单机脚本即可；可容器化
扩展性	中	应用清单化；Agent 扩展位已预留

6.3 组织与实施路径

- 阶段 A（已完成方向）：Web OS 桌面 + 核心财务审计 App + AI 调度
- 阶段 B：客户试点（会计师事务所/企业内部审计）— 模板化底稿、权限与部署文档
- 阶段 C：客户端安装包、私有化、模型本地化、检索集群
- 阶段 D（可选）：Linux 会话/ISO 专用机——仅当市场验证「专用终端」付费意愿

6.4 主要约束

依赖浏览器能力（麦克风、iframe 策略）；公网搜索与外链受目标站限制；审计责任不能外包给模型；多租户强隔离需额外投入。

7. 商业化思考

7.1 价值主张

不是「再做一个财务软件」，而是卖「AI 驱动财务审计工作操作系统」：统一入口、更快上手、可协作、可审计留痕、可私有化。对事务所是提效与标准化；对企业内审是风险可见与流程沉淀；对教育培训是可操练的审计工作台。

7.2 目标客户与场景

- 中小型会计师事务所：底稿导入、初筛、报告草稿、项目协作
- 企业财务/内审部门：往来核查、异常交易初筛、跨部门协同
- 高校/培训机构：实验环境与案例库（学生角色）
- ISV/集成商：私有化交付 + 定制行业规则包

7.3 商业模式选项

模式	收费点	适配
SaaS 订阅	席位 / 公司租户 / 年费	快速获客、持续更新 AI 能力
私有化许可	一次性 License + 年维保	敏感数据、央国企、事务所内网

用量加购	模型 Token、存储、检索配额	与 AI 成本对齐
解决方案	实施、规则包、培训认证	提高客单价与粘性
硬件捆绑（远期）	预装 FinanceOS 的审计终端	需阶段 D 验证

7.4 竞争定位

相对传统 ERP/审计软件：更轻、更 AI、更「工作台」。相对通用 ChatGPT：懂财务对象与权限、能开业务应用、能接企业数据。相对 RPA：强调人机共驾与可解释审计流程，而非纯脚本代替。

7.5 定价与包装建议（示意）

- Basic：桌面 + 导入分析 + 仪表盘（适合教学/试用）
- Pro：全财务模块 + 协作 + AI 调度 + 报告
- Enterprise：私有化、SSO、审计日志、本地模型、专属规则包

7.6 增长飞轮

试用桌面体验 → 导入真实底稿看到风险价值 → 团队协作产生数据资产 → AI 在私有语料上更准 → 增购席位与模型配额。关键是把「好看的 OS 壳」尽快接到「可交付的审计结果」上，避免只停留在交互猎奇。

8. 路线图与风险

8.1 近中期路线图

- 近期：稳定桌面体验、空态引导、导入→仪表盘→报告主路径打磨、说明书与演示环境
- 试点：3-5 家客户、规则包行业化、权限与日志增强、Electron 包
- 规模化：私有化交付标准件、本地模型方案、检索与知识库运营
- 探索：评估专用终端/ISO 需求；拓展渠道（行业协会、培训机构）

8.2 风险与缓释

风险	影响	缓释
AI 幻觉导致错误结论	高	强制人工确认、结论免责、关键步骤可追溯
被当成「玩具桌面」	中	演示必须跑通真实数据主路径
大厂通用助手挤占入口	中	深耕财务对象与私有化合规
外网搜索/浏览不稳定	低	站内知识库兜底、代理降级策略
合规与数据出境	高	本地模型与私有化优先卖点

9. 结论与建议

FinanceOS 的产品机会在于：用「操作系统工作台」重构财务审计软件的交互范式，并用 AI 完成应用调度与分析辅助，同时复用已有审计与财务能力快速形成可售卖闭环。

建议坚持三条主线：

- 体验主线：桌面稳定、空态友好、绝不误导性嵌套/乱弹窗

价值主线：导入→风险可见→可导出报告，让客户为结果付费

商业主线：SaaS 获客 + 私有化收钱 + AI 用量与规则包增值

本文档用于统一产品、研发、商务对 FinanceOS 的认知，可随版本迭代修订。具体操作步骤见《FinanceOS 使用说明书》。